Las pruebas de software son una parte importante del ciclo de vida del desarrollo porque permiten detectar errores y comprobar que un programa cumple con los requisitos establecidos. Entre las diferentes técnicas de prueba se encuentran las pruebas estáticas y las pruebas dinámicas. Aunque ambas buscan mejorar la calidad del software, se realizan de maneras diferentes y pueden complementarse para encontrar distintos tipos de problemas.

Una prueba JUnit es una prueba unitaria utilizada principalmente en aplicaciones desarrolladas con Java. JUnit permite crear y ejecutar pruebas para comprobar de manera individual diferentes partes de un programa, como métodos o funciones, y verificar que produzcan los resultados esperados. Por ejemplo, si se desarrolla un método que realiza una operación matemática, JUnit puede utilizarse para comprobar automáticamente que el resultado sea correcto para diferentes valores de entrada. Esto ayuda a detectar errores desde etapas tempranas y facilita volver a probar el código cuando se realizan cambios. Debido a que estas pruebas pueden automatizarse, JUnit es una herramienta útil para mantener la calidad del software a medida que aumenta su complejidad.

Las pruebas dinámicas son aquellas que requieren ejecutar el software para observar su comportamiento. De acuerdo con el International Software Testing Qualifications Board (ISTQB, 2021), las pruebas dinámicas implican la ejecución del software o de un componente del sistema. En este tipo de pruebas se proporcionan diferentes entradas al programa y se comparan los resultados obtenidos con los resultados esperados. Las pruebas unitarias, de integración y funcionales son algunos ejemplos de pruebas dinámicas. Su objetivo es identificar problemas que aparecen durante la ejecución, como resultados incorrectos, errores de funcionamiento o comportamientos inesperados.

Por otro lado, las pruebas estáticas se realizan sin ejecutar el programa. En lugar de observar el comportamiento del software durante su ejecución, se revisan elementos como los requisitos, el diseño, la documentación y el código fuente para identificar posibles errores o inconsistencias. El ISTQB (2021) señala que las pruebas estáticas pueden realizarse mediante revisiones o análisis estático. Por ejemplo, una revisión de código puede encontrar problemas relacionados con la calidad, consistencia o mantenibilidad antes de que el programa sea ejecutado. Esto permite detectar determinados defectos de manera temprana en el proceso de desarrollo.

La principal diferencia entre las pruebas estáticas y dinámicas es, por lo tanto, que las pruebas estáticas analizan los productos del desarrollo sin ejecutar el software, mientras que las pruebas dinámicas requieren ejecutar el programa para comprobar su comportamiento. Las pruebas estáticas pueden aplicarse incluso a productos que no son ejecutables, como documentos de requisitos o diseños, mientras que las pruebas dinámicas necesitan un software que pueda ejecutarse. Según el ISTQB (2021), ambas técnicas son complementarias porque pueden encontrar diferentes tipos de defectos. Por esta razón, no considero que una técnica sea más importante que la otra, sino que ambas aportan beneficios diferentes al proceso de desarrollo.

Realizar pruebas estáticas y dinámicas es importante porque utilizar únicamente una de ellas podría dejar errores sin detectar. Las pruebas estáticas permiten identificar problemas de manera temprana, cuando generalmente son más fáciles de corregir. Las pruebas dinámicas, por su parte, permiten comprobar que el software funciona correctamente cuando se ejecuta y que cumple con los resultados esperados. Al combinar ambas técnicas, se obtiene una evaluación más completa del software y se puede mejorar tanto su calidad como su confiabilidad.

En conclusión, las pruebas estáticas y dinámicas cumplen funciones diferentes pero complementarias dentro del desarrollo de software. Las pruebas estáticas ayudan a detectar

problemas antes de la ejecución, mientras que las dinámicas permiten comprobar el comportamiento real del programa. Herramientas como JUnit facilitan la automatización de pruebas unitarias en Java y ayudan a verificar que las diferentes partes de una aplicación funcionen como se espera. Utilizar ambas estrategias permite desarrollar software más confiable, mantenible y con menos errores.

Referencias

International Software Testing Qualifications Board. (2021). *Standard glossary of terms used in software testing* (Version 3.3). https://www.istqb.org/

JUnit Team. (2026). *JUnit 5 user guide*. https://docs.junit.org/current/user-guide/